

Data structures

One-dimensional arrays, lists and records — the three ways a program groups related data together, and how to tell them apart in an exam.

What this key knowledge covers

This summary distils everything you need for **KK03 — Data structures** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Our worked example for the whole module

Meet **DeckHand** — a phone app a Year 11 student is building for the (made-up) collectible card game **Hexlings**. Everything we cover lives inside this one app, so the three structures stay concrete:

A duel always has **exactly 5 lanes**. Lane 0, lane 1 ... lane 4. The number never changes.

Open a pack and it **grows**; trade a card and it **shrinks**. Could be 12 cards or 1,200.

One card has a **name**, an **attack**, a **cost**, an **element**, and whether it's a **foil** — all different types.

Why not just use lots of variables?

Imagine storing five cards on the board with five separate variables:

This works for 5, but it falls apart the moment you want to loop, count, sort, or have a *changing* number of things. A **data structure** is a container that holds many related values under **one name**, so you can reach them with an **index** or a **field** and process them in a loop.

A **data structure** is an organised way of storing a **collection** of related data under a single identifier so it can be accessed and processed efficiently.

The three you must know for Units 1 & 2 are the **one-dimensional array**, the **list**, and the **record**. The next three sections take them one at a time — each with a 3-D lab you can play with.

One-dimensional array

A **one-dimensional array** is an ordered collection of elements that are **all the same data type**, stored under one name, and reached by a **numeric index**. Its size is **fixed** when you create it.

Every slot holds the **same kind** of value — e.g. all strings, or all integers. (homogeneous)

You declare a length and that's it. A 5-lane board has lanes 0–4. There is **no lane 5**.

Reach any element *directly* by its position: board[0], board[3]. Indexes start at **0**.

A 5-element array has indexes **0,1,2,3,4** — not 1–5. Asking for board[5] is an **“index out of range”** error. Off-by-one bugs come straight from forgetting arrays start at 0.

List

A **list** is also an ordered collection reached by index — but unlike an array its size is **dynamic**. You can **append**, **insert** and **remove** items while the program runs, and it grows or shrinks to fit.

Starts empty or small, then grows on demand. Perfect when you don't know how many items there'll be.

collection.append(card) adds to the end; collection.remove(0) takes one out.

Remove item 0 and everything after it **shifts down** — the old item 1 becomes the new item 0.

Array = fixed size, set at creation. List = dynamic size, changes at run-time. If a question mentions an *unknown or changing* number of items, that's a flag for a **list**. If it's a *known, fixed* count, an **array** fits.

Record

A **record** is different in kind. Instead of many items of the *same* type, it groups together **fields of varying data types** that all describe **one thing**. You reach a value by its **field name** (or its **field index**) — not by hunting through a list.

One field is a string, the next an integer, the next a Boolean. (heterogeneous)

All fields belong to a **single** item — one card, one student, one booking.

Access by name card.attack or by **field index** card[1] — field 0 is name, field 1 is attack...

Describe the differences

This is the half of the dot point students drop marks on. Learn this table cold — every row is a sentence you can write in an exam.

* In many languages a list can technically hold mixed types, but for VCAA the examinable contrast is **fixed (array) vs dynamic (list)** size — that's the difference graded.

1. Same type or mixed types? Mixed → record. Same → array or list. **2. Does the count change?** Fixed → array. Changes → list.

The structures combine — a list of records

Real apps nest structures inside each other. In DeckHand, your **collection** is a **list**, and **each element of that list is a card** — a **record**. That's a *list of records*: the single most common pattern you'll meet in this course.

So collection[0] uses the **list's numeric index** to grab a record, and ["name"] uses the **record's field name** to grab a value. Two access styles, one line — and a perfect thing to point at when an exam says “describe the differences.”

When you build your own small solution, you'll almost always store "a bunch of things, each with several fields" — students with their marks, products with prices, bookings with times. That's a **list (or array) of records**. Recognising the pattern is half the design.

Choose the right structure

Drag each scenario into the structure that fits best — or tap a chip then tap a bin on touch screens. Think: *same/mixed types? fixed/changing count?*

Command-word decoder

The verb tells you how much to write. Tap to expand.

Quick quiz

Instant feedback — pick an option to see if it's right and why.

Practice questions

Five VCAA-style questions, **19 marks**. Try each on paper first, then reveal a weak answer, a strong answer and the marking guide. Mark your own work — your score saves automatically.

An array, because it stores the cards.

Names the structure but the justification is generic — it doesn't mention the feature that makes an array right here.

A **one-dimensional array**. The board always has exactly 5 lanes — a **known, fixed number** of same-type items — so a fixed-size, index-accessed array (`board[0] ... board[4]`) is the appropriate structure.

- 1 mark: identifies a one-dimensional array.
- 1 mark: justification linked to the fixed/known count (or fixed size + same type).

A record is a structure that stores data about something, like a card.

Too vague: no mention of varying data types or field access, and the example doesn't show different types.

A **record** groups together **fields of varying data types** that describe a **single entity**, accessed by **field name or field index**. Example — one card: name (string), attack (integer) and isFoil (Boolean), all stored together as one card.

Common traps & misconceptions

Exam trap

1 "An array can grow when you need more space." No — an array is **fixed** at creation. Fix: that's a list. Use "fixed (array) vs dynamic (list)" to mark the difference.

Exam trap

2 Calling a record "an array of different types." A record holds the **fields of one entity**, accessed by field name — it isn't indexed like an array of many items. Fix: say "one entity, varying data types, field name/index."

Exam trap

3 Indexing from 1. A 5-element array has indexes **0-4**. `board[5]` is out of range. Fix: remember the first element is index 0.

Exam trap

4 “List and array are basically the same.” They share index access, but the graded difference is **size: fixed vs dynamic**. Fix: always name that difference explicitly.

Exam trap

5 Naming a structure but not justifying it. “Use a record” earns little. Fix: link it to the scenario — “a record, because a card has several attributes of different types.”

Exam trap

6 Forgetting the “field index” detail for records. VCAA explicitly lists *varying data types* and *field index*. Fix: mention that a field can be reached by its name or its position (field index).

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Identify • 2 marks

• Justify DeckHand stores the cards on the 5-lane battle board. Identify the most appropriate data structure for the board and justify your choice.

✗ A weak answer

An array, because it stores the cards. Names the structure but the justification is generic — it doesn’t mention the feature that makes an array right here.

✓ A strong answer

A **one-dimensional array**. The board always has exactly 5 lanes — a **known, fixed number** of same-type items — so a fixed-size, index-accessed array (`board[0] ... board[4]`) is the appropriate structure.

How the marks are awarded

- 1 mark: identifies a one-dimensional array.
- 1 mark: justification linked to the fixed/known count (or fixed size + same type).

Q2. Describe · 3 marks

Describe the data structure 'record', and give one example from DeckHand that shows fields of varying data types.

✗ A weak answer

A record is a structure that stores data about something, like a card. Too vague: no mention of varying data types or field access, and the example doesn't show different types.

✓ A strong answer

A **record** groups together **fields of varying data types** that describe a **single entity**, accessed by **field name or field index**. Example — one card: name (string), attack (integer) and isFoil (Boolean), all stored together as one card.

How the marks are awarded

- 1 mark: groups fields about one entity.
- 1 mark: states fields are of varying data types (accessed by field name/index).
- 1 mark: a valid example showing at least two different data types.

Q3. Distinguish · 4 marks

Distinguish between a one-dimensional array and a list. Refer to two clear differences.

✗ A weak answer

An array and a list are both used to store lots of data using an index, so they are very similar. States only the similarity. 'Distinguish' needs the differences, on matched features.

✓ A strong answer

Both use a numeric index, but they differ in **size** and **flexibility**. (1) An array has a **fixed size** set when it is created, whereas a **list is dynamic** and can grow or shrink at run-time. (2) Because of this, you cannot add beyond an array's declared length, while a list supports **append and remove**, re-indexing its items automatically.

How the marks are awarded

- Up to 2 marks: size difference — array fixed vs list dynamic.
- Up to 2 marks: a second valid difference (e.g. add/remove & re-indexing, or cannot exceed declared length).
- Capped at 2 if only similarities or one difference are given.

Q4. Explain · 4 marks

A student decides to store each player's card collection. Explain why a list is more appropriate than a one-dimensional array for this purpose.

✗ A weak answer

A list is better because lists are newer and easier to use than arrays. No cause-and-effect tied to the scenario, and the reasoning is incorrect.

✓ A strong answer

The number of cards in a collection is **not known in advance and changes** as players open packs or trade. A list has a **dynamic size**, so it can **grow and shrink** to match — appending new cards and removing traded ones. An array's size is **fixed** at creation, so it would either run out of space or waste it. Therefore a list suits the changing collection, while an array does not.

How the marks are awarded

- 1 mark: identifies the count is unknown/changing.
- 1 mark: list is dynamic (grows/shrinks, append/remove).
- 1 mark: array is fixed size — limitation for this case.
- 1 mark: clear link concluding the list is more appropriate.